FLEETFLOW

Intelligent Logistics Management Platform

Comprehensive Technical Architecture & Algorithm Reference Guide

Covering Modules 1, 2, & 3: String Algorithms, Suffix Structures, and Advanced Dynamic Programming

Date: October 2026 | Version 2.1

1. Executive Summary & Platform Overview

FleetFlow is an enterprise-grade logistics and supply chain management platform designed to handle large-scale tracking data, delivery document indexing, address auto-correction, fleet route optimization, and operational batch processing.

The platform models complete logistics operations through key domain abstractions:

- **Delivery Vehicles & Drivers:** Fleet tracking, load capacities, driver task allocation.
- **Warehouses & Distribution Centers:** Multi-hub storage networks and segment consolidation.
- **Packages & Customer Orders:** Package codes, priorities, weights, and status tracking.
- **Shipment Documents:** Bills of Lading, Customs Declarations, Delivery Receipts.
- **Logistics Events & Status Logs:** Continuous telemetry streams, delivery status histories.

Layered System Architecture

Layer	Components	Responsibility

UI / CLI Layer	`Main.java`, `FleetFlowApplication.java`	Entry point, command-line arguments (`--demo`, `--m1`, `--m2`, `--m3`), application loop.
Menu Layer	`MainMenu`, `SearchMenu`, `DocumentMenu`, `DPMenu`	Interactive user interface navigation for String Algorithms, Suffix Structures, and Dynamic Programming.
Service Layer	`SearchService`, `DocumentService`, `DPService`	Business logic orchestration, linking DSA algorithms to domain data.
Algorithms Layer	`algorithms.string`, `algorithms.suffix`, `algorithms.dp`	Pure, high-performance Data Structures & Algorithms implementations.
Model & Repository	`Shipment`, `Package`, `ShipmentDocument`, `ShipmentRepository`	Domain entity data holders and sample dataset providers.

2. Module 1 (CO1): String Search Algorithms

Module 1 focuses on high-speed text processing and pattern recognition across shipment IDs, package barcodes, and unstructured status log streams.

2.1 Knuth-Morris-Pratt (KMP) Algorithm

Class: KMP.java **Complexity: O(N + M)**

> **Logistics Use Case: Search Shipment IDs & Delivery Keywords**
>
> KMP avoids redundant comparisons by computing a Longest Prefix Suffix (LPS) array. When a mismatch occurs after matching characters, the algorithm uses the LPS table to skip ahead without backtracking the text index.

> *LPS[i] = Length of the longest proper prefix of pattern[0..i] that is also a suffix of pattern[0..i].*

```
// Example Usage in SearchService.java
List<Shipment> results = KMP.searchKeywords(shipments, "New York");
// Time Complexity: O(N + M) where N = text length, M = query pattern length.
// Space Complexity: O(M) for the LPS array.
```

2.2 Z-Function (Z-Algorithm)

Class: ZAlgorithm.java **Complexity: O(N)**

> **Logistics Use Case: Detect Repeated Shipment Patterns & Route Loops**
>
> Computes a Z-array where `Z[i]` is the length of the longest common prefix between string `S` and the suffix of `S` starting at index `i`.

> *If n % k == 0 and Z[k] == n - k, then prefix of length k is the fundamental repeating unit of sequence S.*

```
// Detects repeating hub route sequences like "HUB1-HUB2-HUB1-HUB2-HUB1-HUB2"
ZAlgorithm.RepeatAnalysis analysis = ZAlgorithm.analyzeRepetitions(routeSequence);
```

```
System.out.println(analysis.toString());
// Output: "Fully Periodic Pattern: 'HUB1-HUB2-' repeated 3 times."
```

2.3 Rabin-Karp Algorithm

Class: RabinKarp.java **Complexity: Avg O(N + M)**

> **Logistics Use Case: Efficient Package Code Rolling Hash Matching**
>
> Uses a rolling hash function to slide over the text in $O(1)$ constant time per window shift, performing character comparisons only when hash values match.

> *Hash(t[i+1 .. i+m]) = (BASE * (Hash(t[i .. i+m-1]) - t[i] * BASE^(m-1)) + t[i+m]) mod PRIME*

```
// Search package codes like "PKG-DELUXE-9901" across manifest blocks
List<Shipment> matches = RabinKarp.searchPackageCodes(shipments, "DELUXE");
```

2.4 Aho-Corasick Automaton

Class: AhoCorasick.java **Complexity: O(N + Z)**

> **Logistics Use Case: Multi-Pattern Delivery Status Search**
>
> Constructs a Trie with BFS failure links to scan unstructured telemetry stream logs for multiple status patterns (e.g. `IN_TRANSIT`, `DELAYED`, `CUSTOMS_HOLD`, `DELIVERED`) simultaneously in a single linear pass.

```
AhoCorasick ac = new AhoCorasick(Arrays.asList("IN_TRANSIT", "DELAYED", "DELIVERED"));
List<StatusMatch> matches = ac.search(logStreamText);
// Time Complexity: O(N + Z) where N is log stream length, Z is total match occurrences.
```

3. Module 2 (CO2): Suffix Structures & Document Similarity

Module 2 manages large-scale document archives, logistics contract indexing, repeated address extraction, and fast substring queries across Bills of Lading and clearance manifests.

3.1 Suffix Array & SA-IS Algorithm

Class: SuffixArray.java **SA-IS: O(N) Linear Time**

> **Logistics Use Case: Document Indexing & Binary Search Pattern Query**
>
> Stores sorted starting indices of all suffixes of a text document. FleetFlow implements the state-of-the-art **SA-IS (Suffix Array Induced Sorting)** algorithm to build the suffix array in linear $O(N)$ time.

```
// Create Suffix Array via SA-IS for a Bill of Lading document
SuffixArray sa = new SuffixArray(billOfLadingContent);

// Search occurrences of query "Boston" via Binary Search in O(M log N) time
int[] positions = sa.searchAllOccurrences("Boston");
```

3.2 Kasai's LCP (Longest Common Prefix) Algorithm

Class: KasaiLCP.java **Complexity: O(N) Linear Time**

> **Logistics Use Case: Detect Repeated Addresses & Contract Boilerplate**
>
> Computes the LCP array using the inverse suffix array (rank array) in $O(N)$ linear time. High LCP values reveal duplicated customer addresses, shared terms, and boilerplate instructions across documents.

> *LCP[rank[i]] >= LCP[rank[i-1]] - 1*

```
KasaiLCP.LCPResult result = KasaiLCP.computeLCP(documentStream, sa.getSuffixArray());
System.out.println("Longest Duplicate Text: " + result.getLongestRepeatedSubstring());
```

3.3 Suffix Trees

Class: SuffixTree.java **Query: O(M) Time**

> **Logistics Use Case: Fast Substring Pattern Matching**
>
> Stores all suffixes in a compressed Trie structure, enabling sub-string verification and key index location in time proportional only to the pattern length $O(M)$, independent of document size.

```
SuffixTree tree = new SuffixTree(manifestText);
List<Integer> matchIndices = tree.search("Handle with care");
```

4. Module 3 (CO3): Advanced Dynamic Programming & Optimization

Module 3 provides mathematical optimization tools for address correction, optimal fleet routing, operation sequence consolidation, and frequency-weighted key lookups.

4.1 Levenshtein & Damerau-Levenshtein Edit Distance

Class: DamerauLevenshtein.java / WagnerFischer.java **Complexity: O(M * N)**

> **Logistics Use Case: Fuzzy Address Typo Correction**
>
> Computes minimum edit operations (Insertions, Deletions, Substitutions, and Transposition Swaps) to match customer input addresses against a database of verified street addresses.

> *DP[i][j] = min(DP[i-1][j]+1, DP[i][j-1]+1, DP[i-1][j-1]+cost, DP[i-2][j-2]+1 (if transposed))*

```
// Input: "500 Frieght Wy, Boston, MA 02110"
AddressMatchResult match = DamerauLevenshtein.correctAddress(typoInput, addressDatabase);
// Result: Corrected to "500 Freight Way, Boston, MA 02110" (Similarity: 93.9%)
```

4.2 Bitmask DP / Held-Karp TSP

Class: BitmaskTSP.java **Complexity: O(N^2 * 2^N)**

> **Logistics Use Case: Multi-Stop Fleet Delivery Route Optimization**
>
> Solves the Travelling Salesperson Problem for N delivery hubs using bitmask state representation `DP(mask, u)`, reducing runtime from brute-force $O(N!)$ to exponential DP $O(N^2 \cdot 2^N)$.

```
// Computes optimal closed-loop delivery tour visiting Chicago, NY, Boston, PHL, WDC
BitmaskTSP.TSPResult route = BitmaskTSP.solveTSP(distanceMatrix, hubNames);
System.out.println(route.getOptimalHubSequence());
```

4.3 Matrix-Chain Multiplication DP

Class: MatrixChainMultiplication.java / IntervalDP.java **Complexity: O(N^3)**

> **Logistics Use Case: Contiguous Logistics Operations Sequence Optimization**
>
> Determines the optimal parenthesization order for batching contiguous hub processing stages (e.g. Unloading $\rightarrow$ Sorting $\rightarrow$ Inspection $\rightarrow$ Consolidation) to minimize operational cost.

$$m[i, j] = min_{i <= k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} * p_k * p_j \}$$

```
MatrixChainResult result = MatrixChainMultiplication.solveMatrixChain(dimensions, opNames
// Optimal Execution: ((Unloading -> Sorting) -> ((Inspection -> Consolidation) -> Dispatc
```

4.4 Optimal Binary Search Tree (OBST)

Class: OptimalBST.java **Complexity: O(N^3)**

> **Logistics Use Case: Frequently Accessed Shipment Keys Optimization**
>
> Constructs a binary search tree that minimizes expected search time based on key request probabilities p_i and unsuccessful search probabilities q_i.

```
OptimalBST.OBSTResult obst = OptimalBST.computeOptimalBST(keys, probabilitiesP, probabili
System.out.println("Minimum Expected Search Cost: " + obst.getMinExpectedCost());
```

5. Summary of DSA Algorithm Capabilities

Module	Algorithm	Primary Logistics Application	Time Complexity	Space Complexity
Module 1 (String Alg)	KMP	Shipment ID & Delivery Keyword Search	$O(N + M)$	$O(M)$
	Z-Algorithm	Route Loop & Periodicity Detection	$O(N)$	$O(N)$
	Rabin-Karp	Package Barcode Rolling Hash Search	Avg $O(N + M)$	$O(1)$
	Aho-Corasick	Multi-Pattern Status Log Stream Scanner	$O(N + Z)$	$O(\Sigma \cdot M)$
Module 2 (Suffix Struct)	Suffix Array	Document Description Index & Binary Search	$O(M \log N)$ search	$O(N)$
	SA-IS Algorithm	Linear-Time Large Document Indexing	$O(N)$ linear build	$O(N)$
	Kasai LCP	Repeated Address & Boilerplate Detection	$O(N)$	$O(N)$
	Suffix Tree	Fast Substring Matching across Archives	$O(M)$ query	$O(N)$
Module 3 (Advanced DP)	Damerau-Levenshtein	Transposition-Aware Address Typo Correction	$O(M \cdot N)$	$O(M \cdot N)$
	Bitmask DP (TSP)	Multi-Stop Fleet Closed-Loop Routing	$O(N^2 \cdot 2^N)$	$O(N \cdot 2^N)$
	Matrix-Chain DP	Batch Logistics Operations Consolidation	$O(N^3)$	$O(N^2)$
	Optimal BST	High-Frequency Key Search Tree Optimization	$O(N^3)$	$O(N^2)$

6. Execution & Testing Instructions

6.1 Compiling the Project

```
# Open PowerShell in workspace root:
& "C:\Users\munmu\.jdks\openjdk-26.0.2\bin\javac.exe" -d bin (Get-ChildItem -Recurse -Filt
```

6.2 Running Algorithm Demonstrations

```
# Run full demonstration suite across Modules 1, 2, & 3:
& "C:\Users\munmu\.jdks\openjdk-26.0.2\bin\java.exe" -cp bin Main --demo

# Run individual module demonstration suites:
& "C:\Users\munmu\.jdks\openjdk-26.0.2\bin\java.exe" -cp bin Main --m1  # String Search
& "C:\Users\munmu\.jdks\openjdk-26.0.2\bin\java.exe" -cp bin Main --m2  # Suffix Structure
& "C:\Users\munmu\.jdks\openjdk-26.0.2\bin\java.exe" -cp bin Main --m3  # Advanced DP
```

6.3 Running Interactive CLI Application

```
& "C:\Users\munmu\.jdks\openjdk-26.0.2\bin\java.exe" -cp bin Main
```

FleetFlow Intelligent Logistics Platform © 2026. All Rights Reserved.